

Цель работы: научиться работать с множеством потоков в C#.

Выполнение заданий:

Вариант 11

Задание 1: Создать 4 потока, каждый из которых будет выполнять функцию WriteString. Назначить потокам приоритеты. Запустить все четыре потока, передав в качестве параметра их номера.

Листинг или ссылка на проект:

```
using System;
using System.Threading;

namespace Thread_Step_1
{
    class Program
    {
        static void WriteString(object _Data)
        {
            string str_for_out = (string)_Data;
            for (int i = 0; i <= 1000; i++)
                Console.Write(str_for_out);
        }

        static void Main()
        {
            Thread th_1 = new Thread(WriteString);
            Thread th_2 = new Thread(WriteString);
            Thread th_3 = new Thread(WriteString);
            Thread th_4 = new Thread(WriteString);

            th_1.Priority = ThreadPriority.Highest;
            th_2.Priority = ThreadPriority.BelowNormal;
            th_3.Priority = ThreadPriority.Normal;
            th_4.Priority = ThreadPriority.Lowest;

            th_1.Start("1");
            th_2.Start("2");
            th_3.Start("3");
            th_4.Start("4");
            Console.WriteLine("\nВсе потоки запущены ");

            th_1.Join();
            th_2.Join();
            th_3.Join();
            th_4.Join();
            Console.ReadKey();
        }
    }
}
```

Результат работы программы (копия экрана):

[illegible]

Задание 2: Разработка простого многопоточного компонента (пока без потоков)

Листинг или ссылка на проект:

frmCalculations.cs

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Drawing;  
using System.Linq;  
using System.Text;
```

```

using System.Threading.Tasks;
using System.Windows.Forms;

namespace Приложение_Windows
{
    public partial class frmCalculations : Form
    {
        Calculator Calculator1;

        public frmCalculations()
        {
            InitializeComponent();
            Calculator1 = new Calculator();
            Calculator1.FactorialComplete += new
Calculator.FactorialCompleteHandler(this.FactorialHandler);
            Calculator1.FactorialMinusOneComplete += new
Calculator.FactorialCompleteHandler(this.FactorialMinusHandler);
            Calculator1.AddTwoComplete += new
Calculator.AddTwoCompleteHandler(this.AddTwoHandler);
            Calculator1.LoopComplete += new Calculator.LoopCompleteHandler(this.LoopDoneHandler);
        }

        private void btnFactorial1_Click(object sender, EventArgs e)
        {
            Calculator1.varFact1 = int.Parse(txtValue.Text);
            // Disables the btnFactorial1 until this calculation is complete.
            btnFactorial1.Enabled = false;
            Calculator1.Factorial();
        }

        private void btnFactorial2_Click(object sender, EventArgs e)
        {
            Calculator1.varFact2 = int.Parse(txtValue.Text);
            btnFactorial2.Enabled = false;
            Calculator1.FactorialMinusOne();
        }

        private void btnAddTwo_Click(object sender, EventArgs e)
        {
            Calculator1.varAddTwo = int.Parse(txtValue.Text);
            btnAddTwo.Enabled = false;
            Calculator1.AddTwo();
        }

        private void btnRunLoops_Click(object sender, EventArgs e)
        {
            Calculator1.varLoopValue = int.Parse(txtValue.Text);
            btnRunLoops.Enabled = false;
            // Lets the user know that a loop is running
            lblRunLoops.Text = "Looping";
            Calculator1.RunALoop();
        }

        private void FactorialHandler(double Value, double Calculations)
        // Displays the returned value in the appropriate label.
        {
            lblFactorial1.Text = Value.ToString();
            // Re-enables the button so it can be used again.
            btnFactorial1.Enabled = true;
            // Updates the label that displays the total calculations performed
            lblTotalCalculations.Text = "TotalCalculations are " +
                Calculations.ToString();
        }

        private void FactorialMinusHandler(double Value, double Calculations)
        {
            lblFactorial2.Text = Value.ToString();
            btnFactorial2.Enabled = true;
            lblTotalCalculations.Text = "TotalCalculations are " + Calculations.ToString();
        }
    }
}

```

```

    }

    private void AddTwoHandler(int Value, double Calculations)
    {
        lblAddTwo.Text = Value.ToString();
        btnAddTwo.Enabled = true;
        lblTotalCalculations.Text = "TotalCalculations are " +
            Calculations.ToString();
    }

    private void LoopDoneHandler(double Calculations, int Count)
    {
        btnRunLoops.Enabled = true;
        lblRunLoops.Text = Count.ToString();
        lblTotalCalculations.Text = "TotalCalculations are " +
            Calculations.ToString();
    }
}
}

```

Calculator.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Приложение_Windows
{
    public partial class Calculator : Component
    {
        public int varAddTwo;
        public int varFact1;
        public int varFact2;
        public int varLoopValue;
        public double varTotalCalculations = 0;

        // This delegate will be invoked with two of your events.
        public delegate void FactorialCompleteHandler(double Factorial, double
TotalCalculations);
        public delegate void AddTwoCompleteHandler(int Result, double TotalCalculations);
        public delegate void LoopCompleteHandler(double TotalCalculations, int Counter);

        public event FactorialCompleteHandler FactorialComplete;
        public event FactorialCompleteHandler FactorialMinusOneComplete;
        public event AddTwoCompleteHandler AddTwoComplete;
        public event LoopCompleteHandler LoopComplete;

        public Calculator()
        {
            InitializeComponent();
        }

        public Calculator(IContainer container)
        {
            container.Add(this);
            InitializeComponent();
        }

        // This method will calculate the value of a number factorial.
        // (varFact1!)
        public void Factorial()
        {
            double varResult = 1;
            double varTotalAsOfNow = 0;
            for (int varX = 1; varX <= varFact1; varX++)
            {

```

```

        varResult *= varX;
        varTotalCalculations += 1;
        varTotalAsOfNow = varTotalCalculations;
    }
    FactorialComplete(varResult, varTotalAsOfNow);
}

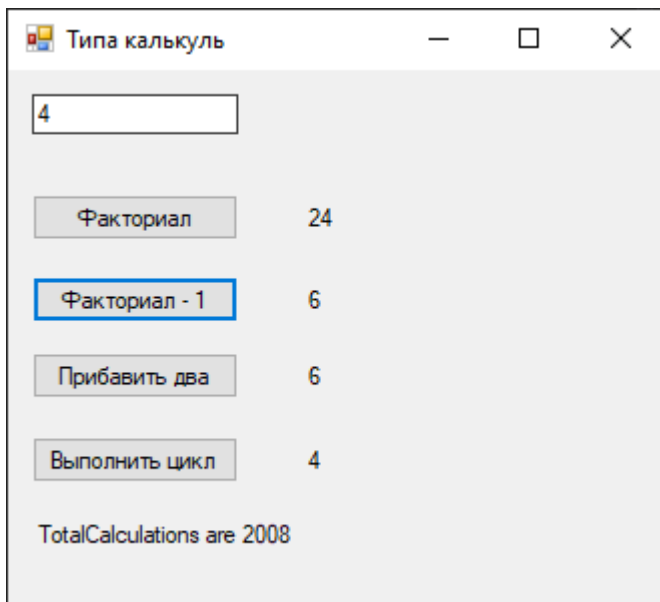
// This method will calculate the value of a number minus 1 factorial
// (varFact2-1!).
public void FactorialMinusOne()
{
    double varTotalAsOfNow = 0;
    double varResult = 1;
    // Performs a factorial calculation on varFact2 - 1.
    for (int varX = 1; varX <= varFact2 - 1; varX++)
    {
        varResult *= varX;
        // Increments varTotalCalculations and keeps track of the current
        // total as of this instant.
        varTotalCalculations += 1;
        varTotalAsOfNow = varTotalCalculations;
    }
    // Signals that the method has completed, and communicates the
    // result and a value of total calculations performed up to this
    // point.
    FactorialMinusOneComplete(varResult, varTotalAsOfNow);
}

// This method will add two to a number (varAddTwo+2).
public void AddTwo()
{
    double varTotalAsOfNow = 0;
    int varResult = varAddTwo + 2;
    varTotalCalculations += 1;
    varTotalAsOfNow = varTotalCalculations;
    AddTwoComplete(varResult, varTotalAsOfNow);
}

// This method will run a loop with a nested loop varLoopValue times.
public void RunALoop()
{
    int varX;
    double varTotalAsOfNow = 0;
    for (varX = 1; varX <= varLoopValue; varX++)
    {
        // This nested loop is added solely for the purpose of slowing down
        // the program and creating a processor-intensive application.
        for (int varY = 1; varY <= 500; varY++)
        {
            varTotalCalculations += 1;
            varTotalAsOfNow = varTotalCalculations;
        }
    }
    LoopComplete(varTotalAsOfNow, varLoopValue);
}
}
}

```

Результат работы программы (копия экрана):



Задание 3: Разработка простого многопоточного компонента

Листинг или ссылка на проект:

frmCalculations.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Приложение_Windows
{
    public partial class frmCalculations : Form
    {
        Calculator Calculator1;
        public delegate void FHandler(double Value, double Calculations);
        public delegate void A2Handler(int Value, double Calculations);
        public delegate void LDHandler(double Calculations, int Count);

        public frmCalculations()
        {
            InitializeComponent();
            Calculator1 = new Calculator();
            Calculator1.FactorialComplete += new
Calculator.FactorialCompleteHandler(this.FactorialHandler);
            Calculator1.FactorialMinusOneComplete += new
Calculator.FactorialCompleteHandler(this.FactorialMinusHandler);
            Calculator1.AddTwoComplete += new
Calculator.AddTwoCompleteHandler(this.AddTwoHandler);
            Calculator1.LoopComplete += new Calculator.LoopCompleteHandler(this.LoopDoneHandler);
        }

        public void FactHandler(double Value, double Calculations)
        {
            lblFactorial1.Text = Value.ToString();
            // Re-enables the button so it can be used again.
            btnFactorial1.Enabled = true;
            // Updates the label that displays the total calculations performed
            lblTotalCalculations.Text = "TotalCalculations are " + Calculations.ToString();
        }
    }
}
```

```

public void Fact1Handler(double Value, double Calculations)
{
    lblFactorial2.Text = Value.ToString();
    btnFactorial2.Enabled = true;
    lblTotalCalculations.Text = "TotalCalculations are " + Calculations.ToString();
}
public void Add2Handler(int Value, double Calculations)
{
    lblAddTwo.Text = Value.ToString();
    btnAddTwo.Enabled = true;
    lblTotalCalculations.Text = "TotalCalculations are " + Calculations.ToString();
}
public void LDoneHandler(double Calculations, int Count)
{
    btnRunLoops.Enabled = true;
    lblRunLoops.Text = Count.ToString();
    lblTotalCalculations.Text = "TotalCalculations are " + Calculations.ToString();
}

private void btnFactorial1_Click(object sender, EventArgs e)
{
    Calculator1.varFact1 = int.Parse(txtValue.Text);
    // Disables the btnFactorial1 until this calculation is complete.
    btnFactorial1.Enabled = false;
    // Calculator1.Factorial();
    Calculator1.ChooseThreads(1);
}

private void btnFactorial2_Click(object sender, EventArgs e)
{
    Calculator1.varFact2 = int.Parse(txtValue.Text);
    btnFactorial2.Enabled = false;
    // Calculator1.FactorialMinusOne();
    Calculator1.ChooseThreads(2);
}

private void btnAddTwo_Click(object sender, EventArgs e)
{
    Calculator1.varAddTwo = int.Parse(txtValue.Text);
    btnAddTwo.Enabled = false;
    // Calculator1.AddTwo();
    Calculator1.ChooseThreads(3);
}

private void btnRunLoops_Click(object sender, EventArgs e)
{
    Calculator1.varLoopValue = int.Parse(txtValue.Text);
    btnRunLoops.Enabled = false;
    // Lets the user know that a loop is running
    lblRunLoops.Text = "Looping";
    // Calculator1.RunALoop();
    Calculator1.ChooseThreads(4);
}

private void FactorialHandler(double Value, double Calculations)
{
    // BeginInvoke causes asynchronous execution to begin at the address
    // specified by the delegate. Simply put, it transfers execution of
    // this method back to the main thread. Any parameters required by
    // the method contained at the delegate are wrapped in an object and
    // passed.
    this.BeginInvoke(new FHandler(FactHandler), new Object[]
        {Value, Calculations});
}

private void FactorialMinusHandler(double Value, double Calculations)
{
    this.BeginInvoke(new FHandler(Fact1Handler), new Object[] {Value, Calculations});
}

```

```

        private void AddTwoHandler(int Value, double Calculations)
        {
            this.BeginInvoke(new A2Handler(Add2Handler), new Object[] {Value, Calculations});
        }

        private void LoopDoneHandler(double Calculations, int Count)
        {
            this.BeginInvoke(new LDHandler(LDoneHandler), new Object[] {Calculations, Count});
        }
    }
}

```

Calculator.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Приложение_Windows
{
    public partial class Calculator : Component
    {
        // Declares the variables you will use to hold your thread objects.
        public System.Threading.Thread FactorialThread;
        public System.Threading.Thread FactorialMinusOneThread;
        public System.Threading.Thread AddTwoThread;
        public System.Threading.Thread LoopThread;

        public int varAddTwo;
        public int varFact1;
        public int varFact2;
        public int varLoopValue;
        public double varTotalCalculations = 0;

        // This delegate will be invoked with two of your events.
        public delegate void FactorialCompleteHandler(double Factorial, double
TotalCalculations);
        public delegate void AddTwoCompleteHandler(int Result, double TotalCalculations);
        public delegate void LoopCompleteHandler(double TotalCalculations, int Counter);

        public event FactorialCompleteHandler FactorialComplete;
        public event FactorialCompleteHandler FactorialMinusOneComplete;
        public event AddTwoCompleteHandler AddTwoComplete;
        public event LoopCompleteHandler LoopComplete;

        public Calculator()
        {
            InitializeComponent();
        }

        public Calculator(IContainer container)
        {
            container.Add(this);
            InitializeComponent();
        }

        // This method will calculate the value of a number factorial.
        // (varFact1!)
        public void Factorial()
        {
            double varResult = 1;
            double varTotalAsOfNow = 0;
            for (int varX = 1; varX <= varFact1; varX++)
            {
                varResult *= varX;
            }
        }
    }
}

```



```

        lock (this)
        {
            varTotalCalculations += 1;
            varTotalAsOfNow = varTotalCalculations;
        }
    }
    FactorialComplete(varResult, varTotalAsOfNow);
}

// This method will calculate the value of a number minus 1 factorial
// (varFact2-1!).
public void FactorialMinusOne()
{
    double varTotalAsOfNow = 0;
    double varResult = 1;
    // Performs a factorial calculation on varFact2 - 1.
    for (int varX = 1; varX <= varFact2 - 1; varX++)
    {
        varResult *= varX;
        // Increments varTotalCalculations and keeps track of the current
        // total as of this instant.
        lock (this)
        {
            varTotalCalculations += 1;
            varTotalAsOfNow = varTotalCalculations;
        }
    }
    // Signals that the method has completed, and communicates the
    // result and a value of total calculations performed up to this
    // point.
    FactorialMinusOneComplete(varResult, varTotalAsOfNow);
}

// This method will add two to a number (varAddTwo+2).
public void AddTwo()
{
    double varTotalAsOfNow = 0;
    int varResult = varAddTwo + 2;
    lock (this)
    {
        varTotalCalculations += 1;
        varTotalAsOfNow = varTotalCalculations;
    }
    AddTwoComplete(varResult, varTotalAsOfNow);
}

// This method will run a loop with a nested loop varLoopValue times.
public void RunALoop()
{
    int varX;
    double varTotalAsOfNow = 0;
    for (varX = 1; varX <= varLoopValue; varX++)
    {
        // This nested loop is added solely for the purpose of slowing down
        // the program and creating a processor-intensive application.
        for (int varY = 1; varY <= 500; varY++)
        {
            lock (this)
            {
                varTotalCalculations += 1;
                varTotalAsOfNow = varTotalCalculations;
            }
        }
    }
    LoopComplete(varTotalAsOfNow, varLoopValue);
}

public void ChooseThreads(int threadNumber)
{

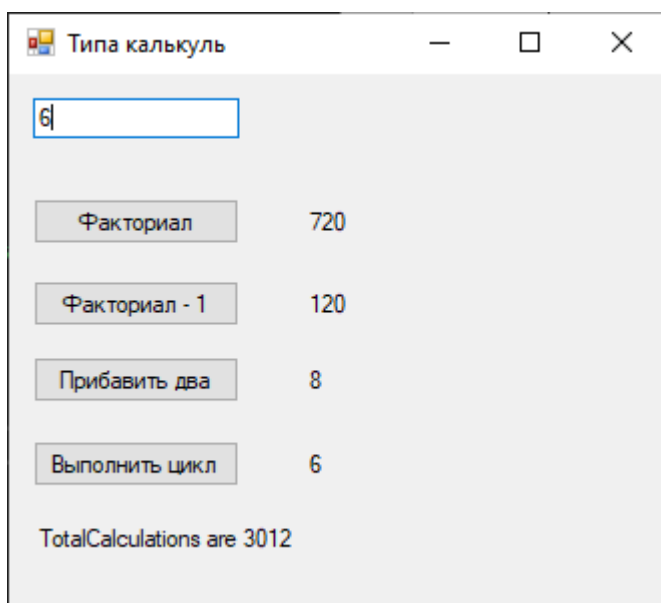
```

```

// Determines which thread to start based on the value it receives.
switch (threadNumber)
{
    case 1:
        // Sets the thread using the AddressOf the subroutine where
        // the thread will start.
        FactorialThread = new System.Threading.Thread(new
            System.Threading.ThreadStart(this.Factorial));
        // Starts the thread.
        FactorialThread.Start();
        break;
    case 2:
        FactorialMinusOneThread = new
            System.Threading.Thread(new
                System.Threading.ThreadStart(this.FactorialMinusOne));
        FactorialMinusOneThread.Start();
        break;
    case 3:
        AddTwoThread = new System.Threading.Thread(new
            System.Threading.ThreadStart(this.AddTwo));
        AddTwoThread.Start();
        break;
    case 4:
        LoopThread = new System.Threading.Thread(new
            System.Threading.ThreadStart(this.RunALoop));
        LoopThread.Start();
        break;
}
}
}
}

```

Результат работы программы (копия экрана):



Контрольные вопросы

1. Продолжите фразу: «Для работы с потоками нам необходимо подключить...»
System.Threading
2. Каким образом создаются потоки?

```
Thread th_1 = new Thread(<Название функции>);
```

Вывод: я научился работать с множеством потоков в C#.